RETROENGINE BETA ENCYCLOPEDIA

Volume 3: Graphics RHI & Pipeline Engineering Guide

1. DUAL-BACKEND RHI ARCHITECTURE & ENGINE ABSTRACTION

RetroEngine decouples high-level scene logic from low-level graphics hardware through the IRenderBackend contract interface. The engine includes two native hardware backends: VulkanBackend for Windows, Linux, and macOS (via MoltenVK), and MetalBackend for macOS (Apple Silicon and Intel Macs). Backend loading is managed automatically by BackendLoader.

RHI Backend Interface	Vulkan 1.3 Backend (VulkanBackend)	Metal 3 Backend (MetalBackend)
Target OS	Windows, Linux, macOS (MoltenVK)	macOS (Apple Silicon & Intel Macs)
Descriptor Model	VK_EXT_descriptor_indexing (Bindless Table)	Metal 3 Argument Buffers (MTLGPUFamilyMetal3)
Rendering Pass Model	Dynamic Rendering (VK_KHR_dynamic_rendering)	Native Apple Render Pass Encoders (MTLRenderCommandEncoder)
Staging / Ring Buffers	VulkanUploadRing / VulkanRingBuffer	MetalRingBuffer (Shared CPU/GPU Unified Memory)
Shader Pipeline	glslc SPIR-V Bytecode Compilation	Runtime MSL (.metal) Source Pipeline Swapping

Stateless Draw Queue & Render Command Dispatch

SceneRenderPipeline generates lock-free double-buffered RenderQueue arrays during the frame tick. Draw commands (RenderCommand) are sorted by pipeline state, material ID, and distance before submission to the active RHI backend.

2. VULKAN 1.3 BACKEND & BINDLESS DESCRIPTOR PIPELINE

The VulkanBackend leverages modern Vulkan 1.3 features to eliminate legacy Vulkan boilerplate. By utilizing VK_KHR_dynamic_rendering, the backend renders directly to swapchain images without creating VkRenderPass or VkFramebuffer objects.

Vulkan Subsystem	Vulkan API Structure / Extension	Implementation Details
Dynamic Rendering	`vkCmdBeginRenderingKHR`	Binds color/depth attachments directly to command buffers.
Bindless Descriptors	`VkDescriptorSetLayoutBinding`	Allocates descriptor array of 4,096 samplers for zero-rebind drawing.
Pipeline Barriers	`VkImageMemoryBarrier2`	Executes layout transitions (UNDEFINED -> COLOR_ATTACHMENT -> PRESENT).
Swapchain	`VulkanSwapchain`	Manages double/triple buffering with VK_PRESENT_MODE_FIFO_KHR.

Bindless Texture Array Indexing

All materials in the scene bind their textures into a single global GPU descriptor array. Shaders access textures using integer indices passed via push constants, eliminating per-draw descriptor set rebinding.

3. METAL 3 BACKEND & APPLE ARGUMENT BUFFERS

The MetalBackend provides a zero-overhead Apple Silicon rendering path using native Metal 3 APIs. It interfaces with macOS system frameworks via an Objective-C JNI bridge (`LibFFI`, `ObjCRuntime`).

Metal Subsystem	Metal API Class	Apple Silicon Optimization
Metal Device	`MTLDevice (MetalDevice)`	Initializes system default Metal device (Apple M1/M2/M3/M4 or Intel).
Argument Buffers	`MTLArgumentEncoder`	Encodes material texture pointers into unified memory argument buffers.
Render Pass Encoder	`MTLRenderCommandEncoder`	Encodes pass commands into MTLCommandBuffer with zero driver overhead.
Unified Memory	`MTLResourceStorageModeShared`	Leverages unified memory architecture (UMA) for zero-copy CPU-GPU data transfers.

> **Zero-Copy Memory Access on Apple Silicon**
>
> On Apple Silicon hardware, host CPU memory and GPU VRAM share the same physical LPDDR RAM pool. MetalBackend uses MTLResourceStorageModeShared to allow GPU shaders to read CPU-written buffers without PCI-e copy passes.

4. VRAM RESOURCE MANAGEMENT & RING BUFFERS

Dynamic uniform updates and vertex staging are backed by triple-buffered upload ring buffers (VulkanUploadRing, MetalRingBuffer). Structure descriptors maintain 16-byte std140/std430 alignment rules.

Resource Descriptor	Allocation Data Structure	VRAM Management Rule
BufferDesc	`BufferDesc.java`	Defines buffer size, usage flags (VERTEX, INDEX, UNIFORM), and memory domain.
TextureDesc	`TextureDesc.java`	Defines width, height, format (RGBA8, DEPTH32F), and mip levels.
PixelBufferData	`PixelBufferData.java`	Holds raw CPU image bytes prior to GPU texture upload.
MeshBufferData	`MeshBufferData.java`	Holds float vertex position, normal, UV, and bone index/weight buffers.

Std140 Uniform Alignment Calculation

Uniform buffer blocks passed to Vulkan/Metal shaders must strictly adhere to 16-byte boundary alignment. RetroEngine automatically pads matrix and vector offsets inside SceneLights uniform streams to prevent GPU memory corruption.

5. MULTI-PASS RENDER PIPELINE & DRAW DISPATCH

The SceneRenderPipeline coordinates scene rendering across five discrete passes: Shadow Depth Pass, Opaque Pass, Skinned Mesh Pass, Infinite Grid Pass, and Game UI Pass.

Pass Sequence	Target Pass & Shaders	Pass Objective & Render State
1. Shadow Depth	`shadow_depth (.vert/.frag)`	Renders scene geometry from directional light space to 2048x2048 depth map.
2. Opaque Lighting	`blinn_phong (.vert/.frag)`	Renders static 3D meshes with Blinn-Phong lighting, shadows, and lightmaps.
3. Skinned Mesh	`blinn_phong_skinned (.vert)`	Renders animated meshes evaluating 64-matrix bone palettes on GPU.
4. Infinite Grid	`infinite_grid (.vert/.frag)`	Renders editor 3D reference grid with fading horizon distance.
5. Game UI Pass	`ui (.vert/.frag)`	Renders 2D quads, cards, pills, progress bars, and font text glyphs.

Frustum Culling & View-Projection Extraction

Before submitting draw commands, FrustumCuller extracts 6 frustum planes using Gribb-Hartmann fast extraction. Entities outside the view frustum are culled instantly on the CPU, preventing unnecessary draw call submission.

6. 2048x2048 SHADOW MAPPING & PCF FILTERING

Shadow rendering is driven by DirectionalShadowFrustum, generating a 2048x2048 Depth32Float shadow map. Shaders execute Percentage-Closer Filtering (PCF) to produce smooth anti-aliased shadow edges.

Shadow Technique	Shader Implementation	Visual Quality & Performance
Frustum Fitting	`DirectionalShadowFrustum`	Fits light orthographic projection tightly around main camera view frustum.
3x3 PCF Filter	`blinn_phong.frag / .metal`	Samples 9 shadow map texels per fragment, blending depth comparisons.
Slope Bias Correction	`Depth Bias Calculation`	Applies dynamic depth bias (tan(theta)) to eliminate shadow acne artifacts.
Metal Hardware PCF	`sampleCompare`	Uses native Metal hardware shadow sampler for single-cycle PCF filtering.

> **Raycast Ground Shadow Blob Fallback**
> On low-power mobile or integrated GPUs where full 2048x2048 shadow maps are disabled, VehicleSuspensionSolver and CharacterControllerSolver project soft ground shadow blobs under entities as a fallback.

7. SHADER ARCHITECTURE & LIVE HOT-RELOADING

Shaders in RetroEngine are modular. On Vulkan, GLSL source files (.vert / .frag) are compiled to SPIR-V using `glslc`. On Metal, `.metal` shaders compile directly to MSL libraries. Live shader hot-reloading allows modifying shaders without restarting.

Shader Pipeline	Vulkan GLSL File	Metal Shader File	Role & Description
Blinn-Phong Lighting	`blinn_phong.vert/frag`	`blinn_phong.metal`	Main 3D lighting shader with diffuse, specular, and PCF shadows.
Skeletal Skinning	`blinn_phong_skinned.vert`	`blinn_phong.metal`	Evaluates 64-bone matrix arrays for animated skeletal meshes.
Infinite Grid	`infinite_grid.vert/frag`	`blinn_phong.metal`	Renders infinite editor floor grid with distance fading.
2D Game UI	`ui.vert/frag`	`ui.metal`	Renders 2D quads, rounded cards, progress bars, and text glyphs.
Color-ID Picking	`entity_id.vert/frag`	`entity_id.metal`	Renders integer Color-IDs for offscreen 3D viewport picking.

> **Zero-Downtime Shader Hot-Reload Failure Protection**
>
> If an edited shader fails to compile during hot-reloading, ShaderSources logs the compiler error to EngineConsolePanel and keeps the previously working pipeline active. The engine never crashes due to shader syntax errors.

8. UI & 2D BATCH RENDERER ENGINE

The GameUIRenderer and UIGeometryBatch systems provide zero-allocation 2D batch rendering. UI elements (buttons, cards, progress bars, text) are batched into dynamic primitive vertex buffers.

UI Geometry Primitive	Batch Method	Rendering Characteristics
Filled Quad	`addQuad()`	Batches 4 vertices (2 triangles) with color and UV mapping.
Rounded Card	`addRoundedCard()`	Batches rounded rectangle card with procedural corner radii.
Pill Switch	`addPill()`	Batches pill-shaped capsule UI element for toggles and status indicators.
Text Glyph	`addText()` / `addBoundedText()`	Batches bitmap font glyphs from atlas using pre-compiled UIFont metrics.

Single-Pass UI Batch Consolidation

All UI elements across the entire editor and gameplay HUD are consolidated into a single UIGeometryBatch buffer. The entire UI renders in a single draw call per frame, guaranteeing 60 FPS UI performance.

9. COLOR-ID PICKING & DEBUG RENDER PASS

3D Viewport entity selection uses an offscreen Color-ID render pass. Entities render into a 32-bit integer framebuffer encoding entity IDs into RGB channels. Additionally, ColliderDebugRenderer draws immediate-mode 3D wireframes in viewport space.

Render Target / Overlay	Color Encoding Scheme	Debug Wireframe Color Rules
Scene Entities	`0x00000001 - 0x00FFFFFF`	Active Colliders: Green. Selected Colliders: Cyan. Trigger Colliders: Yellow.
Gizmo Manipulators	`0xFFFE0001 - 0xFFFE0003`	X Axis: Red, Y Axis: Green, Z Axis: Blue.
Skeleton Joint Handles	`0x80000000 + Joint Index`	Bone Joints: Teal spheres. Selected Joint: Yellow highlight.
Character Controller	`Ground Feet Ring & Probe Line`	Grounded: Green. Airborne: Red/Orange.
Vehicle Suspension	`4-Wheel Suspension Rays`	Grounded Ray: Green. Airborne Ray: Red.

Collider Debug Wireframe Color Convention

In the Viewport, all active non-trigger colliders render as Green wireframes. When selected, they turn Cyan. All trigger volumes (isTrigger == true) render as bright Yellow wireframes regardless of selection state.

10. STANDALONE BUILD EXPORTER & COOKER ENGINE

The StandaloneExporter and RpackCooker engines package projects into standalone native binaries. Assets are baked into swizzled, compressed `.rpack` archives, and bundled with an embedded JDK 25 runtime.

Target Platform	Executable Binary Format	Packaging Method
Windows	`GameExecutable.exe`	Bundles Windows JDK 25 runtime + game .jar into standalone launcher executable.
macOS	`GameApp.app (App Bundle)`	Bundles macOS JDK 25 runtime + native Metal shaders into macOS App Bundle.
Linux	`GameLauncher (ELF Binary)`	Bundles Linux JDK 25 runtime + Vulkan libraries into ELF executable.

> **Summary & Beta Roadmap Completion**
>
> This completes Volume 3 of the RetroEngine Beta Encyclopedia. Refer to Volume 4 (Beta QA Testing & Field Guide) for complete verification protocols, memory leak profiling, and edge-case testing field guides.